

mongoDB



INTRODUCTION

Giới thiệu

- ❖ MongoDB là một NoSQL document-oriented database.
- ❖ Thay vì lưu dữ liệu trong bảng (rows/columns) như SQL, MongoDB lưu dữ liệu dưới dạng document (giống JSON).
- ❖ Đặc điểm:
 - Schema-less: linh hoạt, không bắt buộc cấu trúc cố định.
 - Scalable: hỗ trợ sharding để mở rộng theo chiều ngang.
 - Replication: đảm bảo high availability.
 - Query mạnh: hỗ trợ filter, aggregation, indexing.



Database, Collection, Document

Database

Collection

Document

Document

Document

Collection

Document

Document

Document

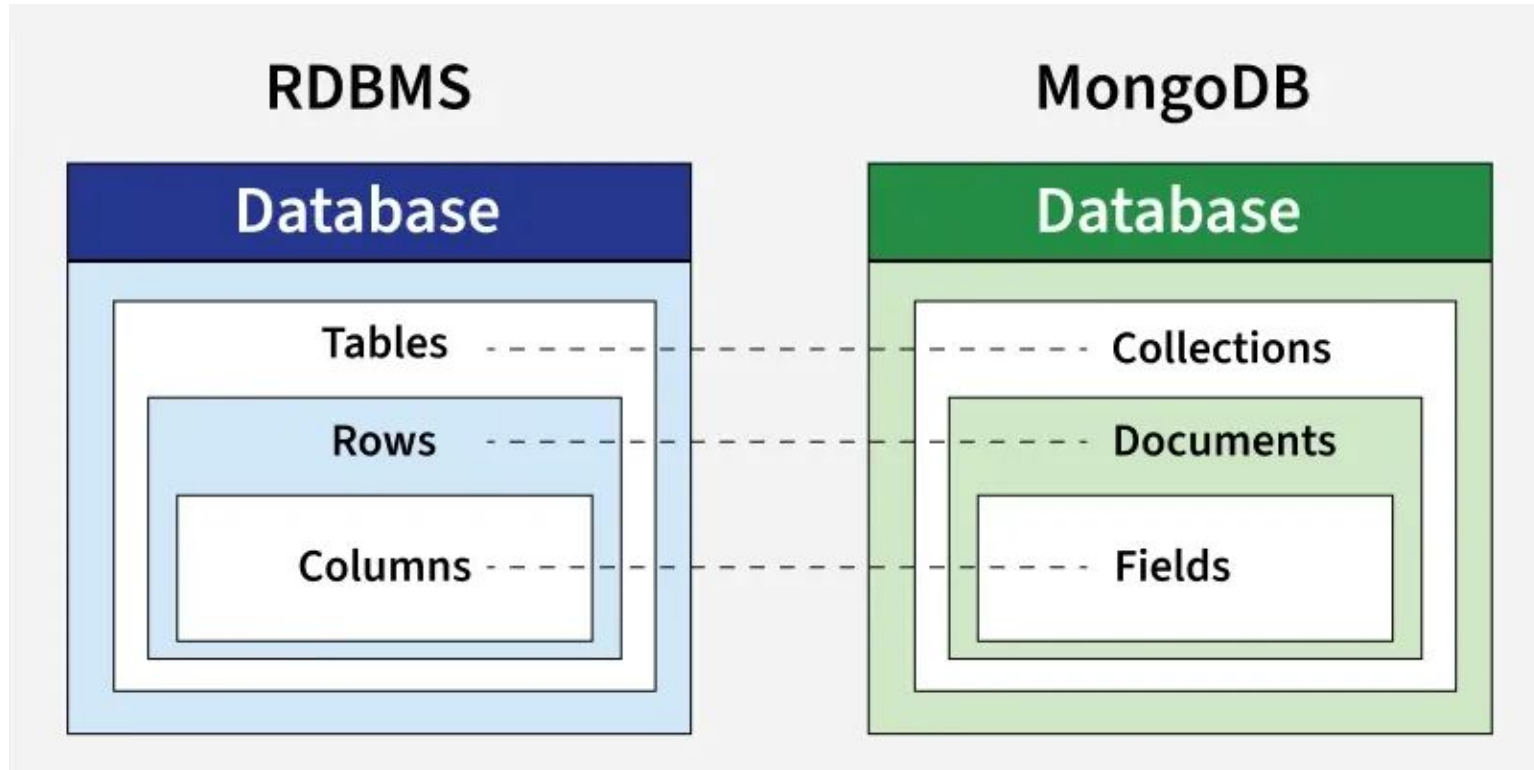
Collection

Document

Document

Document

SQL vs NoSQL



Ví dụ

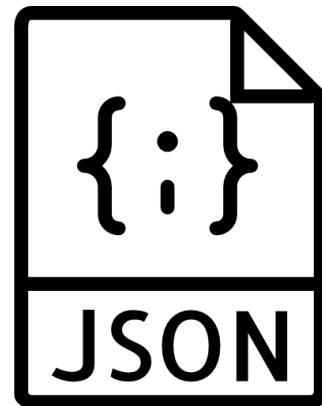


```
{
  _id: "CS2021001",
  name: "Nguyễn Văn An",
  department: "CS",
  year: 3,
  gpa: 3.75,
  email: "an.nguyen@student.edu.vn",
  phone: "0901234567",
  address: {
    street: "123 Nguyễn Huệ",
    district: "Quận 1",
    city: "TP.HCM"
  },
  courses: [
    { code: "CS301", name: "Data Structures", credits: 3, grade: 8.5 },
    { code: "CS302", name: "Algorithms", credits: 3, grade: 9.0 },
    { code: "CS303", name: "Database Systems", credits: 4, grade: 8.8 }
  ],
  skills: ["Java", "Python", "MongoDB", "React"],
  projects: [
    {
      name: "E-commerce Website",
      tech: ["React", "Node.js", "MongoDB"],
      status: "completed",
      score: 9.2
    }
  ],
  scholarships: ["Academic Excellence 2023"],
  isActive: true,
  enrollDate: ISODate("2021-09-01")
}
```

JSON (JavaScript Object Notation)

- ❖ Là định dạng text-based để trao đổi dữ liệu.
- ❖ Rất phổ biến trong web và API.

```
{  
  "name": "Alice",  
  "age": 25,  
  "isStudent": false,  
  "skills": ["MongoDB", "Node.js"]  
}
```



BSON (Binary JSON)



- ❖ Định dạng mà MongoDB thực sự dùng để lưu trữ
- ❖ Nhanh khi encode/decode.
- ❖ Giàu kiểu dữ liệu hơn JSON.
- ❖ Khi insert document bằng JSON, MongoDB sẽ convert sang BSON trước khi lưu.

BSON Data types



- ❖ Ngoài các kiểu trong JSON, BSON hỗ trợ thêm nhiều kiểu dữ liệu khác.
 - ObjectId: ID mặc định trong MongoDB (12-byte).
 - Date: thời gian chính xác tới milliseconds.
 - Int32, Int64, Double, Decimal128: nhiều loại số khác nhau.
 - Binary data: lưu dữ liệu nhị phân (file, hash, ...).
 - Regex: biểu thức chính quy.
 - Timestamp: khác với Date, chủ yếu dùng trong replication.
 - Null, Boolean, Array, Object: giống JSON.

BSON Data types



```
{  
  _id: ObjectId("651d2f2b4f1a2c1234567890"), // ObjectId  
  name: "Alice", // String  
  age: NumberInt(25), // Int32  
  salary: NumberDecimal("1234.56"), // Decimal128  
  createdAt: ISODate("2025-09-18T10:00:00Z") // Date  
}
```

BASIC

CRUD Basic



Create

`insertOne(data, options)`

`insertMany(data, options)`

Read

`find(filter, options)`

`findOne(filter, options)`

Update

`updateOne(filter, data, options)`

`updateMany(filter, data, options)`

`replaceOne(filter, data, options)`

Delete

`deleteOne(filter, options)`

`deleteMany(filter, options)`

Chú ý khi thêm dữ liệu

- ❖ key phải ở dạng nằm trong " "
- ❖ value có thể là string, number, boolean, nested object
- ❖ Tất cả các document insert vào đều có unique ID, type là ObjectId
- ❖ **_id** có tính tăng dần theo thời gian
- ❖ **_id** có thể thủ công thêm cũng được, nhưng cần thận trọng

Chuẩn bị dữ liệu

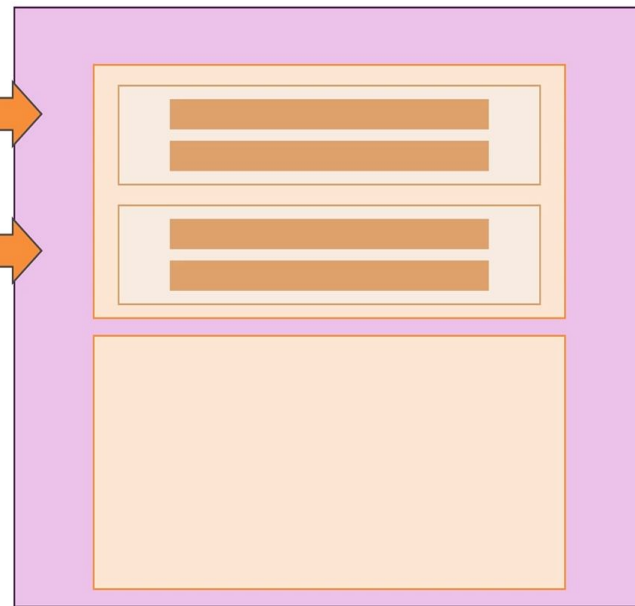
```
db.sinhvien.insertMany([
  {maSV: "SV001", hoTen: "Nguyen Van A", tuoi: 20, lop: "CNTT01", diem: 8.5},
  {maSV: "SV002", hoTen: "Tran Thi B", tuoi: 21, lop: "CNTT02", diem: 7.8},
  {maSV: "SV003", hoTen: "Le Van C", tuoi: 19, lop: "CNTT01", diem: 9.2},
  {maSV: "SV004", hoTen: "Pham Thi D", tuoi: 20, lop: "CNTT03", diem: 6.5},
  {maSV: "SV005", hoTen: "Hoang Van E", tuoi: 22, lop: "CNTT02", diem: 8.0},
  {maSV: "SV006", hoTen: "Vu Thi F", tuoi: 21, lop: "CNTT01", diem: 7.3},
  {maSV: "SV007", hoTen: "Dao Van G", tuoi: 20, lop: "CNTT03", diem: 8.7},
  {maSV: "SV008", hoTen: "Bui Thi H", tuoi: 19, lop: "CNTT02", diem: 9.0},
  {maSV: "SV009", hoTen: "Ngo Van I", tuoi: 21, lop: "CNTT01", diem: 7.1},
  {maSV: "SV010", hoTen: "Dang Thi K", tuoi: 20, lop: "CNTT03", diem: 8.3},
  {maSV: "SV011", hoTen: "Mai Van L", tuoi: 22, lop: "CNTT02", diem: 6.9},
  {maSV: "SV012", hoTen: "Do Thi M", tuoi: 19, lop: "CNTT01", diem: 8.8},
  {maSV: "SV013", hoTen: "Ly Van N", tuoi: 21, lop: "CNTT03", diem: 7.5},
  {maSV: "SV014", hoTen: "Trinh Thi O", tuoi: 20, lop: "CNTT02", diem: 9.1},
  {maSV: "SV015", hoTen: "Duong Van P", tuoi: 22, lop: "CNTT01", diem: 6.8},
  {maSV: "SV016", hoTen: "Cao Thi Q", tuoi: 19, lop: "CNTT03", diem: 8.4},
  {maSV: "SV017", hoTen: "Ta Van R", tuoi: 21, lop: "CNTT02", diem: 7.7},
  {maSV: "SV018", hoTen: "Lo Thi S", tuoi: 20, lop: "CNTT01", diem: 8.9},
  {maSV: "SV019", hoTen: "Phi Van T", tuoi: 22, lop: "CNTT03", diem: 7.2},
  {maSV: "SV020", hoTen: "Lam Thi U", tuoi: 19, lop: "CNTT02", diem: 8.6},
  {maSV: "SV021", hoTen: "Ton Van V", tuoi: 21, lop: "CNTT01", diem: 6.7},
  {maSV: "SV022", hoTen: "Ha Thi W", tuoi: 20, lop: "CNTT03", diem: 9.3},
  {maSV: "SV023", hoTen: "Kieu Van X", tuoi: 22, lop: "CNTT02", diem: 7.4},
  {maSV: "SV024", hoTen: "Mac Thi Y", tuoi: 19, lop: "CNTT01", diem: 8.1},
  {maSV: "SV025", hoTen: "Quan Van Z", tuoi: 21, lop: "CNTT03", diem: 7.9},
  {maSV: "SV026", hoTen: "Tong Thi AA", tuoi: 20, lop: "CNTT02", diem: 8.2},
  {maSV: "SV027", hoTen: "Ung Van BB", tuoi: 22, lop: "CNTT01", diem: 6.6},
  {maSV: "SV028", hoTen: "Vo Thi CC", tuoi: 19, lop: "CNTT03", diem: 9.4},
  {maSV: "SV029", hoTen: "Xa Van DD", tuoi: 21, lop: "CNTT02", diem: 7.6},
  {maSV: "SV030", hoTen: "Yem Thi EE", tuoi: 20, lop: "CNTT01", diem: 8.0}
])
```

Nested Document

```
{
  _id: ObjectId("..."),
  maSV: "SV001",
  hoTen: "Nguyen Van A",
  tuoi: 20,
  thôngTinCaNhan: {
    diaChi: "123 Le Loi, Q1, TP.HCM",
    sdt: "0901234567",
    email: "nguyenvana@email.com",
    phuHuynh: {
      hoTenCha: "Nguyen Van B",
      hoTenMe: "Tran Thi C",
      sdtLienLac: "0987654321",
      ngheNghiep: "Kinh doanh"
    }
  },
}
```

Up to 100 Levels
of Nesting

Max. 16mb /
document



InsertOne()



```
db.students.insertOne({
  _id: ObjectId(),
  studentId: "SV001",
  name: "Nguyễn Văn An",
  email: "an.nguyen@email.com",
  age: 20,
  major: "Công nghệ thông tin",
  gpa: 3.5,
  hobbies: ["đọc sách", "chơi game", "du lịch"],
  address: {
    street: "123 Nguyễn Huệ",
    city: "Hồ Chí Minh",
    district: "Quận 1"
  },
  createdAt: new Date()
})
```


InsertMany()



```
db.sinhvien.insertMany([
  {maSV: "SV001", hoTen: "Nguyen Van A", tuoi: 20, lop: "CNTT01", diem: 8.5},
  {maSV: "SV002", hoTen: "Tran Thi B", tuoi: 21, lop: "CNTT02", diem: 7.8},
  {maSV: "SV003", hoTen: "Le Van C", tuoi: 19, lop: "CNTT01", diem: 9.2},
  {maSV: "SV004", hoTen: "Pham Thi D", tuoi: 20, lop: "CNTT03", diem: 6.5},
  {maSV: "SV005", hoTen: "Hoang Van E", tuoi: 22, lop: "CNTT02", diem: 8.0},
])
```

findOne()



```
db.students.findOne({studentId: "SV001"})
```

findMany()



```
db.students.find({major: "Công nghệ thông tin"})
```

```
db.students.find({  
  age: {$gte: 20},  
  gpa: {$gte: 3.5}  
})
```

SCHEMAS & RELATIONS

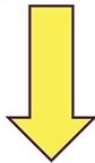
Schema trong NoSQL???



Isn't MongoDB all about having **NO** data Schemas?



MongoDB enforces no schemas! Documents don't have to use the same schema inside of one collection



But that does not mean that you can't use some kind of schema!

Schema trong NoSQL???

Chaos!



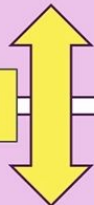
SQL World!

Products

```
{  
  "title": "Book",  
  "price": 12.99  
}
```

Very Different!

```
{  
  "name": "Bottle",  
  "available": true  
}
```

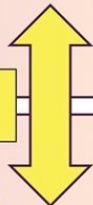


Products

```
{  
  "title": "Book",  
  "price": 12.99  
}
```

Extra Data

```
{  
  "title": "Bottle",  
  "price": 5.99  
  "available": true  
}
```

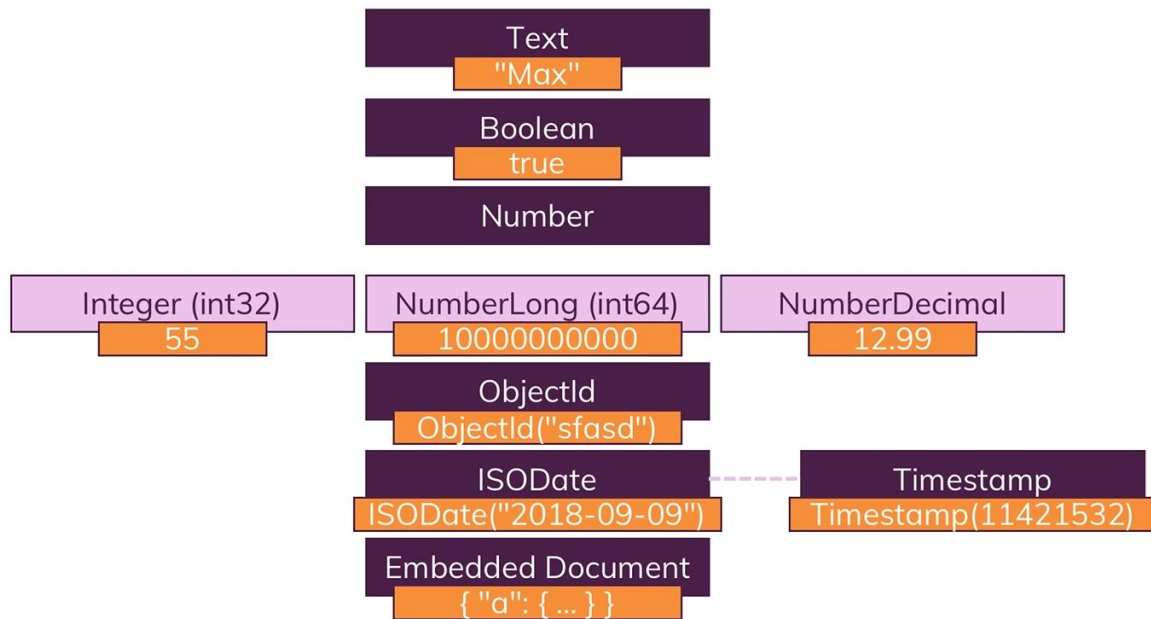


Products

```
{  
  "title": "Book",  
  "price": 12.99  
}
```

```
{  
  "title": "Bottle",  
  "price": 5.99  
}
```

Data Types



Nếu dùng shell (shell được build dựa trên JS), mà trong JS thì chỉ có type number (không phân biệt integer và floating point) nên mặc định là floating. Nhưng mongo có hỗ trợ lưu dạng số int32, int64, numberDecimal để lưu số thực

Chỉ định type khi sử dụng Number



```
db.students.insertOne({ student_id: NumberInt(1), age: NumberInt(20) })
```

```
db.students.insertOne({ student_id: NumberLong(10000000000), age: NumberInt(22) })
```

```
db.students.insertOne({ gpa: NumberDecimal("3.75") })
```


Relations

Nested / Embedded Documents

Customers

```
{
  userName: 'max',
  age: 29,
  address: {
    street: 'Second Street',
    city: 'New York'
  }
}
```

References

Lots of data duplication!

```
{
  user: 'max',
  favBooks: [{...}, {...}]
}
```

```
{
  userName: 'max',
  favBooks: ['id1', 'id2']
}
```

```
{
  _id: 'id1',
  name: 'Lord of the Rings 1'
}
```

“Quan hệ 1-1”

- ❖ 1 bệnh nhân có 1 lịch sử bệnh án

```
db.patients.insertOne({
  _id: ObjectId(),
  name: "Nguyen Van A",
  age: 30,
  medicalHistory: {
    bloodType: "O+",
    allergies: ["penicillin"],
    chronicDiseases: ["diabetes"]
  }
})
```

1 person sở hữu 1 car, nhưng hệ thống muốn thống kê thông tin của user và car

```
db.persons.insertOne({
  _id: ObjectId("person1"),
  name: "Le Van B",
  car_id: ObjectId("car1")
})
db.cars.insertOne({
  _id: ObjectId("car1"),
  brand: "Toyota",
  model: "Camry",
  year: 2020
})
```

“Quan hệ 1-N”

1 question có nhiều answer

```
db.questions.insertOne({
  title: "SQL?",
  content: "SQL viết tắt của từ gì",
  answers: [
    {
      _id: ObjectId(),
      author: "Expert1",
      content: "Structure Query Language",
    },
    {
      _id: ObjectId(),
      author: "Expert2",
      content: "Static Query Language",
    }
  ]
})
```

1 khoa có nhiều sinh viên

```
db.departments.insertOne({
  _id: ObjectId("dept1"),
  name: "Computer Science",
  head: "Dr. Smith"
})

db.students.insertMany([
  {
    _id: ObjectId(),
    name: "Student A",
    department_id: ObjectId("dept1")
  },
  {
    _id: ObjectId(),
    name: "Student B",
    department_id: ObjectId("dept1")
  }
])
```

“Quan hệ N-N”



❖ Cách 1: Dùng Mảng references

```
// students
{
  _id: ObjectId("..."),
  name: "Alice",
  enrolledCourses: [ ObjectId("c1"), ObjectId("c2") ]
}

// courses
{
  _id: ObjectId("c1"),
  title: "Database Systems",
  students: [ ObjectId("...Alice"), ObjectId("...Bob") ]
}
```

“Quan hệ N-N”



- ❖ Cách 2: Lưu thêm reference kèm snapshot nếu cần

```
{
  _id: ObjectId("o2"),
  customer: "Tran Thi B",
  createdAt: ISODate("2025-09-18"),
  products: [
    { productId: ObjectId("p1"), quantity: 1 },
    { productId: ObjectId("p2"), quantity: 2 }
  ]
}
```

“Quan hệ N-N”

- ❖ Cách 3: Tạo collections trung gian trong trường hợp muốn lưu metadata

```
db.students.insertOne({
  _id: ObjectId("student1"),
  name: "Bob"
})

db.courses.insertOne({
  _id: ObjectId("course1"),
  name: "Database Systems"
})
```

```
db.enrollments.insertMany([
  {
    student_id: ObjectId("student1"),
    course_id: ObjectId("course1"),
    grade: 85, // Metadata
    enrolledAt: new Date(), // Metadata
    status: "completed" // Metadata
  }
])
```

Khi nào nên dùng Nested và References



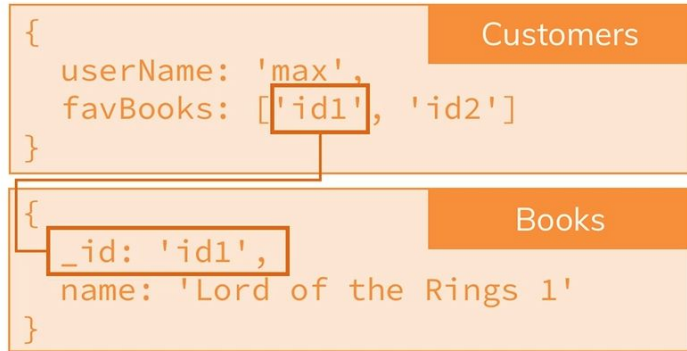
Embedded

- ❖ Dữ liệu có quan hệ chặt chẽ (1-1, 1-N)
- ❖ Thường query cùng lúc
- ❖ Dữ liệu con không thay đổi thường xuyên
- ❖ Không vượt quá giới hạn 16MB

References

- ❖ Dữ liệu độc lập, cần query riêng
- ❖ Quan hệ N-N phức tạp
- ❖ Cần tránh duplicate dữ liệu
- ❖ Dữ liệu thay đổi thường xuyên

Look up



```
customers.aggregate([
  { $lookup: {
    from: "books",
    localField: "favBooks",
    foreignField: "_id"
    as: "favBookData"
  }
}]
```

```
db.students.aggregate([
  {
    $lookup: {
      from: "departments",
      localField: "department",
      foreignField: "_id",
      as: "khoa"
    }
  }
])
```

```
khoa: [
  {
    _id: 'ChE',
    name: 'Chemical Engineering',
    fullName: 'Khoa Hóa học',
    dean: 'PGS.TS Hoàng Thị E',
    established: 1980
  }
]
```

Luôn trả về list



Bài tập



1. users, posts, comments
2. Bill và tickets

CREATE OPERATIONS

Insert



`insertOne()`

```
db.collectionName.insertOne({field: "value"})
```

`insertMany()`

```
db.collectionName.insertMany([
    {field: "value"},
    {field: "value"}])
```

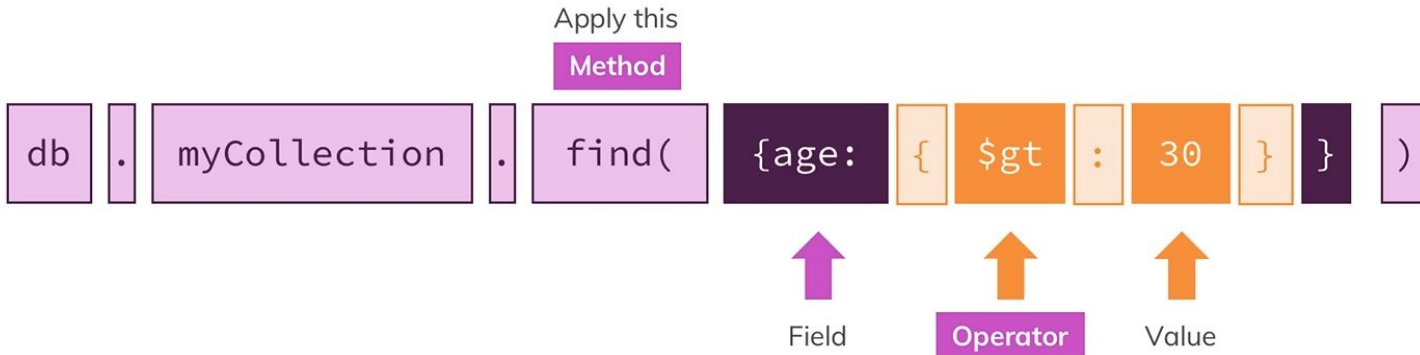
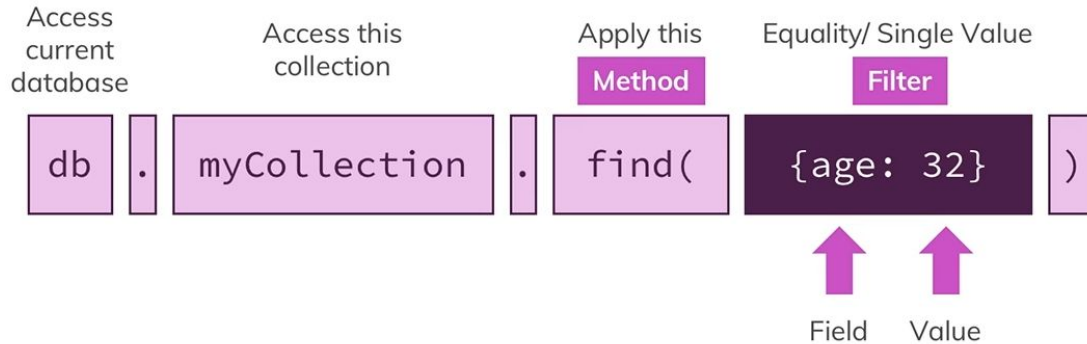
Lưu ý



1. Ordered insert, các document muốn insert sẽ được thực hiện độc lập nhưng chỉ cần 1 cái bị lỗi thì sẽ dừng quá trình đăng sau lại, không roll back document đã insert thành công (mặc định là bật Ordered insert)
2. {ordered: false} thì khi 1 document lỗi thì vẫn tiếp tục, không dừng lại

READ OPERATIONS

Read operations



Projection

- ❖ Projection là cách chọn lọc những field nào sẽ được trả về trong kết quả query, giúp giảm lượng dữ liệu truyền tải và cải thiện hiệu suất.

```
db.students.find({}, {  
  name: 1,  
  department: 1  
})  
  
// Loại bỏ field _id  
db.students.find({}, {  
  name: 1,  
  department: 1,  
  _id: 0  
})
```

```
db.students.find({}, {  
  courses: 0,  
  projects: 0  
})  
// Lấy thông tin nested object  
db.students.find({}, {  
  name: 1,  
  "address.city": 1,  
  "address.district": 1  
})
```

Compare

Toán tử	Ý nghĩa	Ví dụ
<code>\$eq</code>	Bằng (=)	<code>{ price: { \$eq: 100 } }</code> → tìm document có <code>price = 100</code>
<code>\$ne</code>	Khác (!=)	<code>{ status: { \$ne: "done" } }</code> → tìm document có <code>status</code> $\neq$ "done"
<code>\$gt</code>	Lớn hơn (>)	<code>{ age: { \$gt: 18 } }</code> → tìm document có <code>age > 18</code>
<code>\$gte</code>	Lớn hơn hoặc bằng (>=)	<code>{ age: { \$gte: 18 } }</code> → tìm document có <code>age ≥ 18</code>
<code>\$lt</code>	Nhỏ hơn (<)	<code>{ age: { \$lt: 30 } }</code> → tìm document có <code>age < 30</code>
<code>\$lte</code>	Nhỏ hơn hoặc bằng (<=)	<code>{ age: { \$lte: 30 } }</code> → tìm document có <code>age ≤ 30</code>
<code>\$in</code>	Trong tập hợp	<code>{ status: { \$in: ["new", "processing"] } }</code> → tìm document có <code>status</code> là "new" hoặc "processing"
<code>\$nin</code>	Không thuộc tập hợp	<code>{ category: { \$nin: ["electronics", "toys"] } }</code> → tìm document có <code>category</code> không nằm trong 2 giá trị này

Logical operators

```
db.students.find({
  $or: [
    { gpa: { $gt: 3.8 } },
    { scholarships: { $ne: [] } }
  ]
})
```

// AND với điều kiện phức tạp

```
db.students.find({
  $and: [
    { department: "CS" },
    { year: { $gte: 3 } },
    { gpa: { $gte: 3.5 } }
  ]
})
```

// NOT: Sinh viên không có projects

```
db.students.find({
  projects: {
    $not: { $exists: true }
  }
})
```

// NOR: Sinh viên không thuộc CS và không có GPA > 3.8

```
db.students.find({
  $nor: [
    { department: "CS" },
    { gpa: { $gt: 3.8 } }
  ]
})
```

Logical operators



```
db.scores.insertMany([  
  { name: "A", mid_term: 90, final: 80 },  
  { name: "B", mid_term: 70, final: 85 },  
  { name: "C", mid_term: 60, final: 60 }  
])
```

```
db.collection.find({  
  $expr: { $gt: ["$mid_term", "$final"] }  
})
```

Sorting



- ❖ 1 → sort tăng dần (ascending)
- ❖ -1 → sort giảm dần (descending)

```
db.collection.find(query).sort({ field1: 1, field2: -1, field3: 1 })
```

```
db.users.find().sort({ score: -1, name: 1 })
```

Distinct



```
// Lấy danh sách các khoa
```

```
db.students.distinct("department")
```

```
// Lấy danh sách các năm học
```

```
db.students.distinct("year")
```

```
// Lấy danh sách skills unique
```

```
db.students.distinct("skills")
```

```
// Lấy danh sách thành phố
```

```
db.students.distinct("address.city")
```

Skip, Limit



```
db.users.find().skip(10).limit(5)
```

Xử lý trên Array

```
db.orders.find({ "products.name": "Laptop Dell" })
```

```
db.orders.find({  
  products: { $elemMatch: { name: "Laptop Dell", price: { $gt: 1400 } } }  
})
```

```
db.orders.find({ products: { $size: 2 } })
```

```
db.orders.insertMany([  
  {  
    _id: 1,  
    customer: "Nguyen Van A",  
    products: [  
      { name: "Laptop Dell", price: 1500, quantity: 1 },  
      { name: "Chuột Logitech", price: 20, quantity: 2 }  
    ]  
  },  
  {  
    _id: 2,  
    customer: "Tran Thi B",  
    products: [  
      { name: "iPhone 14", price: 1200, quantity: 1 },  
      { name: "Tai nghe Sony", price: 100, quantity: 1 }  
    ]  
  },  
  {  
    _id: 3,  
    customer: "Le Van C",  
    products: [  
      { name: "Laptop Dell", price: 1400, quantity: 1 },  
      { name: "Bàn phím cơ", price: 80, quantity: 1 },  
      { name: "Chuột Logitech", price: 25, quantity: 1 }  
    ]  
  }  
])
```

UPDATE OPERATIONS

\$set



```
// Set đơn giản
db.students.updateOne(
  { _id: "CS2021001" },
  { $set: { lastLogin: new Date() } }
)
```

```
// Set multiple fields
db.students.updateOne(
  { _id: "CS2021001" },
  {
    $set: {
      gpa: 3.85,
      updatedAt: new Date(),
      "profile.bio": "Computer Science
Student"
    }
  }
)
```


\$unset



```
db.students.updateOne(  
  { _id: "CS2021001" },  
  {  
    $unset: {  
      oldEmail: "",  
      temporaryStatus: "",  
      "metadata.temp": ""  
    }  
  }  
)
```

\$inc



```
db.students.updateOne(  
  { _id: "CS2021001" },  
  {  
    $inc: {  
      gpa: 0.05,  
      "stats.loginCount": 1,  
      "stats.projectsCompleted": 1  
    }  
  }  
)
```

\$mul



```
db.students.updateOne(  
  { _id: "CS2021001" },  
  { $mul: { gpa: 1.05 } }  
)
```

Update trên Array

```
// Thêm một skill mới
db.students.updateOne(
  { _id: "CS2021001" },
  { $push: { skills: "Docker" } }
)
```

```
// Thêm một course mới
db.students.updateOne(
  { _id: "CS2021001" },
  {
    $push: {
      courses: {
        code: "CS305",
        name: "Software",
        credits: 4,
        grade: 8.7
      }
    }
  }
)
```

Update trên Array



```
// Thêm nhiều skills cùng lúc
db.students.updateOne(
  { _id: "CS2021001" },
  {
    $push: {
      skills: {
        $each: ["Kubernetes", "AWS", "Terraform"]
      }
    }
  }
)
```

Update trên Array



```
// Thêm skill nếu chưa tồn tại
db.students.updateOne(
  { _id: "CS2021001" },
  { $addToSet: { skills: "Python" } } // Không thêm nếu đã có
)
```

Update trên Array

```
// Thêm skill nếu chưa tồn tại
db.students.updateOne(
  { _id: "CS2021001" },
  { $addToSet: { skills: "Python" } } // Không thêm nếu đã có
)
```

```
// Thêm nhiều skills unique
db.students.updateOne(
  { _id: "CS2021001" },
  {
    $addToSet: {
      skills: {
        $each: ["React", "Node.js", "MongoDB"]
      }
    }
  }
)
```

Update xóa phần tử

```
// Xóa skill cuối cùng
db.students.updateOne(
  { _id: "CS2021001" },
  { $pop: { skills: 1 } } // 1 = cuối, -1 = đầu
)
```

```
// Xóa scholarship đầu tiên
db.students.updateOne(
  { _id: "CS2021001" },
  { $pop: { scholarships: -1 } }
)
```


Upsert



```
// Tạo mới nếu không tồn tại, update nếu có
db.students.updateOne(
  { _id: "CS2024001" },
  {
    $set: {
      name: "Trần Văn Mới",
      department: "CS",
      year: 1,
      gpa: 3.0,
      enrollDate: new Date()
    }
  },
  { upsert: true }
)
```

DELETE OPERATIONS

Delete / Drop



```
db.orders.insertMany([
  { _id: 1, customer: "A", status: "pending" },
  { _id: 2, customer: "B", status: "completed" },
  { _id: 3, customer: "C", status: "pending" }
])
```

// Xóa 1 đơn hàng pending

```
db.orders.deleteOne({ status: "pending" })
```

// Xóa tất cả đơn hàng pending

```
db.orders.deleteMany({ status: "pending" })
```

// Xóa toàn bộ collection

```
db.orders.drop()
```